

A browser assistant that can see your screen — without your personal data leaving your machine

Aavaran, by Team Vagabonds · Project brief · Harsh Bajpai · 17 September 2026

Built for **Problem Statement SIH26171**, set by **ISRO / Department of Space** for Smart India Hackathon 2026 — “On-device Visual Perception for Light-weight Browser Agents” (Software · Smart Automation).

The problem

An AI assistant is only useful if it can see what is on your screen. But your screen holds your PAN, your Aadhaar number, your bank details and an open password field. Today the choice is binary: accept an assistant that sends everything it sees to a company's servers, or have no assistant at all.

For a government department, a bank or a defence establishment this is not a preference — it is a **policy bar**. The technology is barred from precisely the workflows where it would be most valuable.

What I built

A browser extension for **Chrome and Firefox**, paired with a small server that runs on the same machine. Working software, not a prototype sketch:

- A compact vision model runs **inside the browser** and reads the page.
- Every PAN, Aadhaar number, card number, password and human face is replaced with a typed placeholder — **<PII_PAN_1> — before any network request is made.**
- A larger open-weight language model reasons over the censored page and returns a single instruction, such as “click Submit”.
- The browser validates that instruction against its own safety rules, then carries it out.
- A **Privacy Ledger** shows the user exactly what was withheld and the exact bytes that were transmitted, so the claim can be inspected rather than believed.

The reasoning model is **open-weight and runs locally**. The system works with the network disconnected. No data is sent to any external provider at any point.

THE CENTRAL IDEA — WHY THIS IS A GUARANTEE RATHER THAN A PROMISE

The store of real values lives in one isolated component. The component that talks to the network **has never held a real value at any point in the program's execution**.

So “no personal data leaves the machine” is a property of the **architecture**, not an assurance about the quality of my code. A defect in the networking layer *cannot* leak a PAN, because that layer has never been given one. This is the difference between a system that is trustworthy by construction and one that is trustworthy by inspection.

Measured results

Every figure below is produced by a single command from the repository. None is an estimate, and none is rounded in our favour.

Measure	Development set	Unseen set (held out)	Real websites
Ability to read the page correctly	100%	100%	100%
Sensitive-data detection — recall	100%	100%	100%
Sensitive-data detection — precision	100%	100%	100%
Context preserved (not over-redacted)	98%	100%	100%
Personal values leaked	0	0	0

Speed and cost on the machine. About **4.7 seconds** per step on a deliberately slowed low-end laptop (measured under six-times processor throttling, not estimated); roughly 3.7 seconds on a current machine. Around 47 KB is transmitted per step. A complete task — read the form, fill it, submit it — runs **34.5 seconds over six steps in a real browser**, for **50 MB** of browser memory. Those last two are measured inside the browser itself rather than inferred from the server process, because they are 35% of the competition's marking scheme.

Coverage. Field labels are recognised in **eleven Indian languages**, not only English — including Hindi, Marathi, Tamil, Telugu, Bengali, Gujarati, Kannada, Malayalam, Punjabi and Odia.

How I know the numbers are honest

The columns that matter are the second and third. A set of test pages was written **before any tuning began** and has never been optimised against — the discipline a researcher would call a held-out set. It exists because the competition's final evaluation uses websites revealed on the day, so performance on pages I have already seen predicts very little.

The third column answers the harder objection, that I wrote my own examination paper. It scores **ten real pages captured from the live web** — the income-tax portal, RBI, SEBI, UIDAI, MyGov and others. The sensitive values in them are planted and therefore known, but the surrounding markup is not mine: it is the several-thousand-element mess a real site actually serves.

- The three sets are scored separately and reported separately. I quote the unseen figure.
- The system ships its own **counterfactual**: switching off one detection layer drops unseen-set recall to 91.7%. A perfect score is only meaningful if you can show what makes it move.
- **Every safety test is verified to be capable of failing.** I deliberately break the protection and confirm the test turns red before trusting it green — a test that cannot fail is worse than no test, and this project has caught one such test in its own suite.
- Automated tests run on **Linux, macOS and Windows** on every change. **Seventeen checks** without a browser, twenty-four with real browsers and the live model, currently all passing.

Technical summary, for the record

- **Detection** is a three-stage cascade: page structure and accessibility metadata; then pattern matching **with checksum validation** (Verhoeff for Aadhaar, Luhn for payment cards, mod-36 for GSTIN); then person names, via public name lists combined with a dictionary-absence test that catches names no list contains.
- **Checksums matter commercially.** They convert a noisy pattern-matcher into a precise one — which is why a twelve-digit order total that happens to satisfy the Aadhaar checksum is deliberately *not* redacted. Over-redaction destroys the context the assistant needs, and is penalised as heavily as leaking.
- **Faces** are detected and blurred on-device by a 1.2 MB model, verified on pixels to destroy 89% of facial detail with no damage to the surrounding image.
- **Adversarial resistance.** Web pages that attempt to manipulate the assistant into disclosing data are refused structurally: the validator rejects 16 of 24 test instructions, including a genuine data-exfiltration route found and closed during development. The defence does not depend on the AI behaving well — it never possessed a real value to disclose.
- Roughly 6,500 lines of TypeScript and Python. Two tagged releases.

Status and authorship

- **Working today** in both Chrome and Firefox: the complete loop, on-device face blurring, the Privacy Ledger, adversarial-page defences and graceful failure handling.
- Developed as **Aavaran**, the Smart India Hackathon 2026 entry of **Team Vagabonds** — six students of IIM Mumbai. The repository is held in the team's own GitHub organisation; the version history records **79 of 80 commits** as mine, and the design decisions, measurements and test methodology described here are my own work.
- Documented throughout — the architectural decisions are recorded with the measurements that produced them, so the reasoning is auditable and not only the result.

WHY IT MATTERS BEYOND THE COMPETITION

Browser-based AI assistants are currently unusable anywhere the screen is sensitive — government departments, banking, healthcare, defence, and ISRO's own workflows. This work removes that barrier rather than asking anyone to lower it, and the redaction approach is reusable by any assistant, not only this one.